



同濟大學
TONGJI UNIVERSITY

计算机系统结构课程实验 总结报告

实验题目：动态流水线设计与性能定量分析

学号：2352628

姓名：韦畅

指导教师：秦国锋

日期：2025.12.20

一、 实验环境部署与硬件配置说明

- 开发环境：Vivado 2016.2
- 仿真工具：Vivado 内置仿真器
- 开发板：Xilinx NEXYS4 DDR Artix-7 FPGA 开发板

二、 实验的总体结构

2.1 实验设计目标与功能概述

本实验旨在设计并实现一个基于 MIPS 指令集的**动态流水线 CPU**。该设计不仅要求能够正确执行基础的指令流，还必须具备处理复杂指令间相关性的能力。

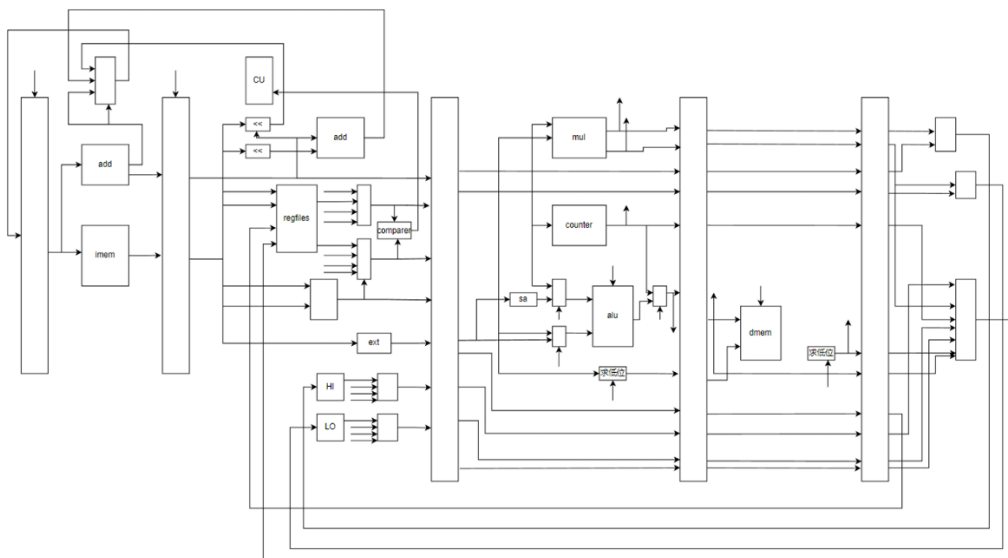
- **指令规模**：本设计共实现了 31 条 MIPS 指令，涵盖了算术逻辑运算、移位、分支跳转及访存等核心类型。
- **硬件交互与中断处理**：系统支持外部异步中断信号。在 CPU 执行复杂的验算程序（如摔鸡蛋问题模型）时，可以通过开发板上的 [按键/拨动开关] 触发暂停中断，并通过七段数码管实时、动态地观测运算过程中的中间值或最终结果。

2.2 动态流水线总体架构

本实验采用经典的五级流水线架构，将指令的执行过程划分为：**取指 (IF)**、**译码 (ID)**、**执行 (EX)**、**访存 (MEM)** 以及**写回 (WB)**。

为了解决流水线中的“冒险”问题，我们在架构中集成了：

1. **旁路前推单元**：通过在 EX 级和 MEM 级引出反馈路径，将尚未写回寄存器堆的数据直接馈送至 ALU 输入端，从而消除 RAW（写后读）数据相关导致的延迟。
2. **流水线暂停逻辑**：当遇到无法通过前推解决的冲突（如 Load-use 冲突）时，自动阻塞 IF 和 ID 级，并插入空操作 (NOP)。
3. **中断控制逻辑**：集成 CPU 协处理器，用于捕获硬件中断并保存断点，实现程序执行状态的保护与恢复。



2.3 指令集定义

为了验证处理器的通用性，本设计选取了具有代表性的指令子集。指令格式严格遵循 MIPS 标准，分为 R 型、I 型和 J 型。

助记符	指令格式					
Bit	31-26	25-21	20-16	15-11	10-6	5-0
add	000000	rs	rt	rd	00000	100000
addu	000000	rs	rt	rd	00000	100001
sub	000000	rs	rt	rd	00000	100010
subu	000000	rs	rt	rd	00000	100011
and	000000	rs	rt	rd	00000	100100
or	000000	rs	rt	rd	00000	100101
xor	000000	rs	rt	rd	00000	100110
nor	000000	rs	rt	rd	00000	100111
slt	000000	rs	rt	rd	00000	101010
sltu	000000	rs	rt	rd	00000	101011
sll	000000	00000	rt	rd	shamt	000000
srl	000000	00000	rt	rd	shamt	000010
sra	000000	00000	rt	rd	shamt	000011
sllv	000000	rs	rt	rd	00000	000100
srlv	000000	rs	rt	rd	00000	000110
srav	000000	rs	rt	rd	00000	000111
jr	000000	rs	00000	00000	00000	001000
addi	001000	rs	rt	immediate (16位)		
addiu	001001	rs	rt	immediate (16位)		
andi	001100	rs	rt	immediate (16位)		
ori	001101	rs	rt	immediate (16位)		
xori	001110	rs	rt	immediate (16位)		
lui	001111	00000	rt	immediate (16位)		
lw	100011	rs	rt	immediate (16位)		
sw	101011	rs	rt	immediate (16位)		
beq	000100	rs	rt	immediate (16位)		
bne	000101	rs	rt	immediate (16位)		
slti	001010	rs	rt	immediate (16位)		
sltiu	001011	rs	rt	immediate (16位)		
j	000010	address (26位)				
jal	000011	address (26位)				
mul	011100	rs	rt	rd	00000	000010

2.4 数据通路与关键控制信号

在总体结构中，数据通路决定了信息的流向，而控制单元（Control Unit）则根据指令类型产生相应的使能信号。

- **ALUOp**: 控制运算器的运算类型；
- **RegWrite**: 决定是否在 WB 级将结果写入目标寄存器；
- **MemToReg**: 控制写回数据是来自存储器还是运算结果；
- **Branch/Jump**: 控制 PC 指针的跳转逻辑。

三、 总体架构部件的解释说明

3.1 动态流水线各阶段模块实现

本设计将 CPU 划分为五个标准流水段，各阶段通过级间寄存器进行数据同步与状态传递。

1. IF（取指）阶段

- **功能描述**: 根据当前 PC 值从指令存储器（IMEM）中读取 32 位指令代码，同时计算 PC+4 为下一条指令做准备。
- **核心组件**:
 - **PC 寄存器**: 受中断信号和 Stall 信号控制，实现程序计数器的跳转、保持或复位。
 - **IMEM (Instruction Memory)**: 存储编译后的机器码。

```
1. module pipe_if(  
2.     input    [31:0] in_pc,  
3.     input    [2:0]  in_pc_sel,  
4.     input    [31:0] in_pc_eaddr,  
5.     input    [31:0] in_pc_baddr,  
6.     input    [31:0] in_pc_raddr,  
7.     input    [31:0] in_pc_jaddr,  
8.     output   [31:0] out_npc,  
9.     output   [31:0] out_pc4,  
10.    output   [31:0] out_instr  
11. );
```

2. ID（译码/读寄存器）阶段

- **功能描述**: 解析指令操作码（Opcode）和功能码（Funct），从寄存器堆（RF）读取源操作数，并扩展立即数。
- **控制单元 (CU)**: 生成所有级间传递的控制信号。
- **冲突检测单元**: 在此阶段判断是否发生 Load-use 冲突，若发生则拉高 Stall 信号。

```
12. module pipe_id(
13.     input          in_clk,
14.     input          in_rst,
15.     input [31:0]   in_pc4,
16.     input [31:0]   in_instr,
17.     input          in_hi_wena,
18.     input          in_lo_wena,
19.     input          in_rd_wena,
20.     input [4:0]    in_rd_waddr,
21.     input [31:0]   in_hi_data,
22.     input [31:0]   in_lo_data,
23.     input [31:0]   in_rd_data,
24.
25.     input [5:0]     in_ex_op,
26.     input [5:0]     in_ex_func,
27.     input [31:0]   in_ex_pc4,
28.     input [31:0]   in_ex_alu_data,
29.     input [31:0]   in_ex_mul_hi,
30.     input [31:0]   in_ex_mul_lo,
31.     input [31:0]   in_ex_div_r,
32.     input [31:0]   in_ex_div_q,
33.     input [31:0]   in_ex_clz_data,
34.     input [31:0]   in_ex_hi_data,
35.     input [31:0]   in_ex_lo_data,
36.     input [31:0]   in_ex_rs_data,
37.     input          in_ex_hi_wena,
38.     input          in_ex_lo_wena,
39.     input          in_ex_rd_wena,
40.     input [1:0]    in_ex_hi_sel,
41.     input [1:0]    in_ex_lo_sel,
42.     input [2:0]    in_ex_rd_sel,
43.     input [4:0]    in_ex_rd_waddr,
44.
45.     input [31:0]   in_mem_pc4,
46.     input [31:0]   in_mem_alu_data,
47.     input [31:0]   in_mem_mul_hi,
48.     input [31:0]   in_mem_mul_lo,
49.     input [31:0]   in_mem_div_q,
50.     input [31:0]   in_mem_div_r,
51.     input [31:0]   in_mem_clz_data,
52.     input [31:0]   in_mem_lo_data,
53.     input [31:0]   in_mem_hi_data,
54.     input [31:0]   in_mem_rs_data,
55.     input [31:0]   in_mem_dmem_data,
```

```
56.    input          in_mem_hi_wena,
57.    input          in_mem_lo_wena,
58.    input          in_mem_rd_wena,
59.    input [1:0]    in_mem_hi_sel,
60.    input [1:0]    in_mem_lo_sel,
61.    input [2:0]    in_mem_rd_sel,
62.    input [4:0]    in_mem_rd_waddr,
63.
64.    output          out_stall,
65.    output          out_branch,
66.    output [5:0]    out_op,
67.    output [5:0]    out_func,
68.    output [2:0]    out_pc_sel,
69.    output [31:0]   out_pc4,
70.    output [31:0]   out_immed,
71.    output [31:0]   out_shamt,
72.    output [31:0]   out_pc_eaddr,
73.    output [31:0]   out_pc_baddr,
74.    output [31:0]   out_pc_jaddr,
75.    output [31:0]   out_pc_raddr,
76.    output [31:0]   out_rs_data,
77.    output [31:0]   out_rt_data,
78.    output [31:0]   out_hi_data,
79.    output [31:0]   out_lo_data,
80.    output [31:0]   out_cp0_data,
81.    output          out_alu_a_sel,
82.    output [1:0]    out_alu_b_sel,
83.    output [3:0]    out_aluc,
84.    output          out_mul_ena,
85.    output          out_div_ena,
86.    output          out_clz_ena,
87.    output          out_mul_sign,
88.    output          out_div_sign,
89.    output          out_hi_wena,
90.    output          out_lo_wena,
91.    output          out_rd_wena,
92.    output          out_cutter_sign,
93.    output          out_cutter_addr_sel,
94.    output [2:0]    out_cutter_sel,
95.    output          out_dmem_ena,
96.    output          out_dmem_wena,
97.    output [1:0]    out_dmem_wsel,
98.    output [1:0]    out_dmem_rsel,
```

```

99.     output [1:0]    out_hi_sel,
100.     output [1:0]    out_lo_sel,
101.     output [2:0]    out_rd_sel,
102.     output [4:0]    out_rd_waddr,
103.     output [31:0]    out_reg6,
104.     output [31:0]    out_reg7,
105.     output [31:0]    out_reg15,
106.     output [31:0]    out_reg16
107. );
108.

```

3. EX（执行）阶段

- 功能描述：执行算术逻辑运算、移位操作或地址计算。
- 核心组件：
 - ALU（算术逻辑单元）：支持加减乘除、逻辑与或、比较等运算。
 - 乘除法器（MUL/DIV）：针对复杂指令设置的独立运算单元。
 - 前推多路选择器：根据 Forward 信号，决定 ALU 输入是来自 ID 级还是 MEM/WB 级的前推数据。

```

1. module pipe_ex(
2.     input          in_rst,
3.     input [31:0]    in_pc4,
4.     input [31:0]    in_immed,
5.     input [31:0]    in_shamt,
6.     input [31:0]    in_rs_data,
7.     input [31:0]    in_rt_data,
8.     input [31:0]    in_hi_data,
9.     input [31:0]    in_lo_data,
10.    input [31:0]    in_cp0_data,
11.    input          in_alu_a_sel,
12.    input [1:0]     in_alu_b_sel,
13.    input [3:0]     in_aluc,
14.    input          in_mul_ena,
15.    input          in_div_ena,
16.    input          in_clz_ena,
17.    input          in_mul_sign,
18.    input          in_div_sign,
19.    input          in_cutter_sign,
20.    input          in_cutter_addr_sel,
21.    input [2:0]     in_cutter_sel,
22.    input          in_dmem_ena,
23.    input          in_dmem_wena,
24.    input [1:0]     in_dmem_wsel,

```

```

25.   input [1:0]    in_dmem_rsel,
26.   input          in_rd_wena,
27.   input          in_hi_wena,
28.   input          in_lo_wena,
29.   input [1:0]    in_hi_sel,
30.   input [1:0]    in_lo_sel,
31.   input [2:0]    in_rd_sel,
32.   input [4:0]    in_rd_waddr,
33.   output [31:0]   out_pc4,
34.   output [31:0]   out_mul_hi,
35.   output [31:0]   out_mul_lo,
36.   output [31:0]   out_div_r,
37.   output [31:0]   out_div_q,
38.   output [31:0]   out_rs_data,
39.   output [31:0]   out_rt_data,
40.   output [31:0]   out_hi_data,
41.   output [31:0]   out_lo_data,
42.   output [31:0]   out_cp0_data,
43.   output [31:0]   out_clz_data,
44.   output [31:0]   out_alu_data,
45.   output          out_cutter_sign,
46.   output          out_cutter_addr_sel,
47.   output [2:0]    out_cutter_sel,
48.   output          out_dmem_ena,
49.   output          out_dmem_wena,
50.   output [1:0]    out_dmem_wsel,
51.   output [1:0]    out_dmem_rsel,
52.   output          out_hi_wena,
53.   output          out_lo_wena,
54.   output          out_rd_wena,
55.   output [1:0]    out_hi_sel,
56.   output [1:0]    out_lo_sel,
57.   output [2:0]    out_rd_sel,
58.   output [4:0]    out_rd_waddr
59. );

```

4. MEM（访存）阶段

- 功能描述：处理存储器读写指令（LW/SW）。
- 核心组件：
 - DMEM (Data Memory)：同步读写的数据存储器。
 - CP0 (协处理器 0)：处理中断嵌套、异常状态字读取及数据输出至外部数码管驱动模块。


```
1. module pipe_mem(
2.     input          in_clk,
3.     input [31:0]   in_pc4,
4.     input [31:0]   in_rs_data,
5.     input [31:0]   in_rt_data,
6.     input [31:0]   in_hi_data,
7.     input [31:0]   in_lo_data,
8.     input [31:0]   in_cp0_data,
9.     input [31:0]   in_alu_data,
10.    input [31:0]   in_mul_hi,
11.    input [31:0]   in_mul_lo,
12.    input [31:0]   in_div_r,
13.    input [31:0]   in_div_q,
14.    input [31:0]   in_clz_data,
15.    input          in_cutter_sign,
16.    input          in_cutter_addr_sel,
17.    input [2:0]    in_cutter_sel,
18.    input [1:0]    in_dmem_wsel,
19.    input [1:0]    in_dmem_rsel,
20.    input          in_dmem_ena,
21.    input          in_dmem_wena,
22.    input          in_hi_wena,
23.    input          in_lo_wena,
24.    input          in_rd_wena,
25.    input [1:0]    in_hi_sel,
26.    input [1:0]    in_lo_sel,
27.    input [2:0]    in_rd_sel,
28.    input [4:0]    in_rd_waddr,
29.
30.    output [31:0]   our_pc4,
31.    output [31:0]   out_rs_data,
32.    output [31:0]   out_hi_data,
33.    output [31:0]   out_lo_data,
34.    output [31:0]   out_cp0_data,
35.    output [31:0]   out_alu_data,
36.    output [31:0]   out_mul_hi,
37.    output [31:0]   out_mul_lo,
38.    output [31:0]   out_div_r,
39.    output [31:0]   out_div_q,
40.    output [31:0]   out_clz_data,
41.    output [31:0]   out_dmem_data,
42.    output          out_hi_wena,
43.    output          out_lo_wena,
44.    output          out_rd_wena,
```

```

45.     output [1:0]    out_hi_sel,
46.     output [1:0]    out_lo_sel,
47.     output [2:0]    out_rd_sel,
48.     output [4:0]    out_rd_waddr
49. );
50.

```

5. WB（写回）阶段

- 功能描述：将运算结果或访存读取的数据写回寄存器堆的目标寄存器。

```

1. module pipe_wb(
2.     input [31:0]    in_pc4,
3.     input [31:0]    in_rs_data,
4.     input [31:0]    in_hi_data,
5.     input [31:0]    in_lo_data,
6.     input [31:0]    in_cp0_data,
7.     input [31:0]    in_alu_data,
8.     input [31:0]    in_mul_hi,
9.     input [31:0]    in_mul_lo,
10.    input [31:0]    in_div_r,
11.    input [31:0]    in_div_q,
12.    input [31:0]    in_clz_data,
13.    input [31:0]    in_dmem_data,
14.    input           in_hi_wena,
15.    input           in_lo_wena,
16.    input           in_rd_wena,
17.    input [1:0]     in_hi_sel,
18.    input [1:0]     in_lo_sel,
19.    input [2:0]     in_rd_sel,
20.    input [4:0]     in_rd_waddr,
21.
22.    output           out_hi_wena,
23.    output           out_lo_wena,
24.    output           out_rd_wena,
25.    output [4:0]     out_rd_waddr,
26.    output [31:0]    out_hi_data,
27.    output [31:0]    out_lo_data,
28.    output [31:0]    out_rd_data
29. );

```

3.2 级间寄存器定义（IF/ID, ID/EX, EX/MEM, MEM/WB）

级间寄存器是“动态”流水线的骨干，用于在时钟上升沿同步传递控制信号和数据。

- 数据传递：确保每一条指令在不同阶段对应的操作数、目的寄存器号同步流动。

- 清空逻辑 (Flush): 当分支预测失败或发生中断时, 通过清零级间寄存器实现流水线冲刷。

3.3 全局控制与冲突处理模块

这是动态流水线区别于单周期 CPU 的关键:

- 前推单元 (Forwarding Unit):
 - 检测 EX/MEM.RegWrite 和 MEM/WB.RegWrite 信号。
 - 当源寄存器号与前序指令的目的寄存器号相等时, 激活数据通路。
- 暂停单元 (Stall Unit):
 - 主要针对 LW 指令引发的 RAW 相关。
 - 实现逻辑:


```
if (ID/EX.MemRead && (ID/EX.RegisterRt == IF/ID.RegisterRs || ID/EX.RegisterRt == IF/ID.RegisterRt))
```

四、实验仿真过程

4.1 仿真策略与测试环境

为了全面验证动态流水线 CPU 的各项功能, 仿真过程分为两个阶段: 单一指令功能验证 (确保每条指令执行正确) 和复杂程序流压力测试 (验证冲突处理逻辑)。

- 仿真工具: Vivado Simulator
- 测试基准: 采用 MIPS 汇编编写的测试程序 (Testbench), 涵盖了 R-type、I-type 和 J-type 指令。

4.2 构建测试平台 (Testbench)

在 Testbench 拨码开关模拟输入中, 主要包含以下核心部分:

1. 时钟信号 (Clk) 与复位信号 (Rst): 生成稳定的工作时钟, 并在仿真开始前对 PC 和寄存器堆进行初始化清零。
2. 指令存储器加载: 通过 readmemh 指令将预先编译好的十六进制机器码加载到 IMEM 模块中。
3. 中断模拟: 通过在仿真脚本中延迟给出一个 Int_Request 信号, 模拟按键产生的暂停请求。

```
1. `timescale 1ns / 1ps
2.
3. module simulation_testbench();
4.     reg          clock_signal, reset_signal, enable_signal;
5.     wire [7:0]    segments, digit_selectors;
6.
7.     // 初始化
8.     initial begin
9.         clock_signal = 1'b0;
10.        reset_signal = 1'b1;
```

```
11.         enable_signal = 1'b1;
12.         #1
13.         reset_signal = 1'b0;
14.     end
15.
16.     // 时钟生成
17.     always begin
18.         #1
19.         clock_signal = ~clock_signal;
20.     end
21.
22.     // 监控信号定义
23.     wire [31:0] program_counter =
simulation_testbench.board_top_module.cpu_module.program_count
er;
24.     wire [31:0] instruction =
simulation_testbench.board_top_module.cpu_module.current_instr
uction;
25.
26.     wire [31:0] register_0 =
simulation_testbench.board_top_module.cpu_module.pipeline_id_s
tage.register_file_module.register_bank[0];
27.     wire [31:0] register_1 =
simulation_testbench.board_top_module.cpu_module.pipeline_id_s
tage.register_file_module.register_bank[1];
28.     wire [31:0] register_2 =
simulation_testbench.board_top_module.cpu_module.pipeline_id_s
tage.register_file_module.register_bank[2];
29.     wire [31:0] register_3 =
simulation_testbench.board_top_module.cpu_module.pipeline_id_s
tage.register_file_module.register_bank[3];
30.     wire [31:0] register_4 =
simulation_testbench.board_top_module.cpu_module.pipeline_id_s
tage.register_file_module.register_bank[4];
31.     wire [31:0] register_5 =
simulation_testbench.board_top_module.cpu_module.pipeline_id_s
tage.register_file_module.register_bank[5];
32.     wire [31:0] register_6 =
simulation_testbench.board_top_module.cpu_module.pipeline_id_s
tage.register_file_module.register_bank[6];
33.     wire [31:0] register_7 =
simulation_testbench.board_top_module.cpu_module.pipeline_id_s
tage.register_file_module.register_bank[7];
```

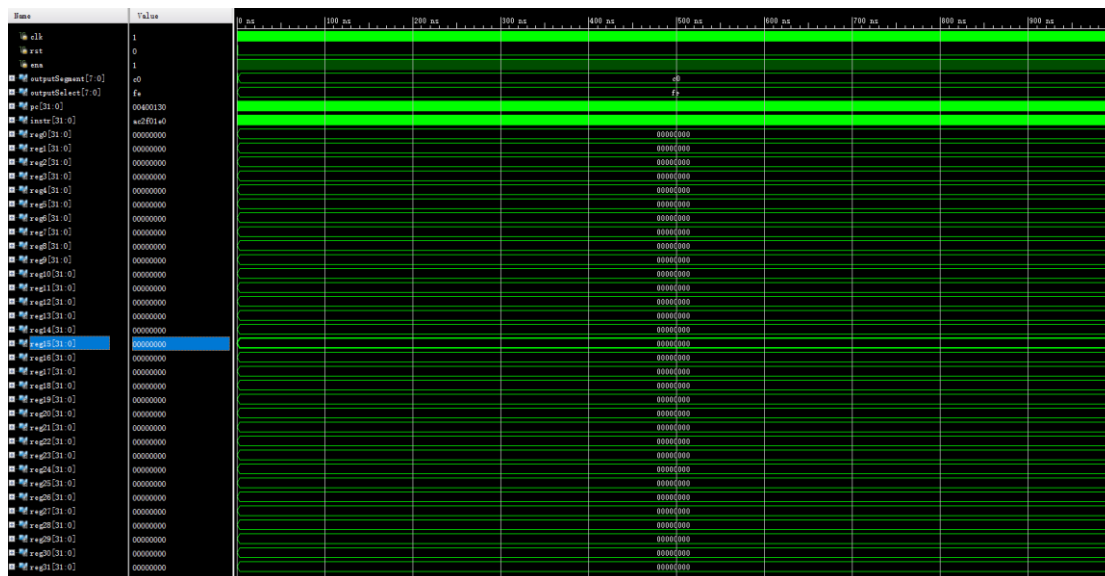
```
34.     wire [31:0] register_8 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[8];
35.     wire [31:0] register_9 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[9];
36.     wire [31:0] register_10 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[10];
37.     wire [31:0] register_11 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[11];
38.     wire [31:0] register_12 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[12];
39.     wire [31:0] register_13 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[13];
40.     wire [31:0] register_14 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[14];
41.     wire [31:0] register_15 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[15];
42.     wire [31:0] register_16 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[16];
43.     wire [31:0] register_17 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[17];
44.     wire [31:0] register_18 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[18];
45.     wire [31:0] register_19 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[19];
46.     wire [31:0] register_20 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[20];
47.     wire [31:0] register_21 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[21];
```

```

48.     wire [31:0] register_22 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[22];
49.     wire [31:0] register_23 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[23];
50.     wire [31:0] register_24 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[24];
51.     wire [31:0] register_25 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[25];
52.     wire [31:0] register_26 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[26];
53.     wire [31:0] register_27 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[27];
54.     wire [31:0] register_28 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[28];
55.     wire [31:0] register_29 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[29];
56.     wire [31:0] register_30 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[30];
57.     wire [31:0] register_31 =
        simulation_testbench.board_top_module.cpu_module.pipeline_id_s
        tage.register_file_module.register_bank[31];
58.
59.     // 实例化顶层模块
60.     board_top board_top_module(
61.         .clk(clock_signal),
62.         .rst(reset_signal),
63.         .ena(enable_signal),
64.         .o_seg(segments),
65.         .o_sel(digit_selectors)
66.     );
67.
68. endmodule

```

五、 实验仿真的波形图



六、 实验验算数学模型及算法程序

6.1 构建测试 c 程序

```

1. #include <iostream>
2. #include <iomanip>
3.
4. class ArrayProcessor
5. {
6. private:
7.     static const int ELEMENT_COUNT = 60;
8.     int arrayA[ELEMENT_COUNT];
9.     int arrayB[ELEMENT_COUNT];
10.    int arrayC[ELEMENT_COUNT];
11.    int arrayD[ELEMENT_COUNT];
12.
13.    // 计算数组 A 和 B 的值
14.    void computeArrayAandB()
15.    {
16.        arrayA[0] = 0;
17.        arrayB[0] = 1;
18.
19.        for (int idx = 1; idx < ELEMENT_COUNT; idx++)
20.        {
21.            arrayA[idx] = arrayA[idx - 1] + idx;
22.            arrayB[idx] = arrayB[idx - 1] + 3 * idx;
23.        }
24.    }

```

```
25.
26. // 根据索引范围计算数组 C 和 D
27. void computeArrayCandD()
28. {
29.     for (int idx = 0; idx < ELEMENT_COUNT; idx++)
30.     {
31.         computeForIndex(idx);
32.     }
33. }
34.
35. // 处理单个索引
36. void computeForIndex(int idx)
37. {
38.     if (idx < 20)
39.     {
40.         handleFirstRange(idx);
41.     }
42.     else if (idx < 40)
43.     {
44.         handleSecondRange(idx);
45.     }
46.     else
47.     {
48.         handleThirdRange(idx);
49.     }
50. }
51.
52. void handleFirstRange(int idx)
53. {
54.     arrayC[idx] = arrayA[idx];
55.     arrayD[idx] = arrayB[idx];
56. }
57.
58. void handleSecondRange(int idx)
59. {
60.     arrayC[idx] = arrayA[idx] + arrayB[idx];
61.     arrayD[idx] = arrayA[idx] * arrayC[idx];
62. }
63.
64. void handleThirdRange(int idx)
65. {
66.     arrayC[idx] = arrayA[idx] * arrayB[idx];
67.     arrayD[idx] = arrayC[idx] * arrayB[idx];
68. }
```



```

69.
70. // 显示结果
71. void displayResults() const
72. {
73.     std::cout << "数组元素值如下: " << std::endl;
74.     std::cout << "索引   数组 A       数组 B       数组 C       数组
D" << std::endl;
75.
76.     for (int idx = 0; idx < ELEMENT_COUNT; idx++)
77.     {
78.         displayRow(idx);
79.     }
80. }
81.
82. void displayRow(int idx) const
83. {
84.     std::cout << "i=" << std::setw(2) << idx << "   "
85.         << formatHex(arrayA[idx]) << "   "
86.         << formatHex(arrayB[idx]) << "   "
87.         << formatHex(arrayC[idx]) << "   "
88.         << formatHex(arrayD[idx]) << std::endl;
89. }
90.
91. std::string formatHex(int value) const
92. {
93.     std::stringstream ss;
94.     ss << std::hex << std::setw(8) << std::setfill('0') <<
value;
95.     return ss.str();
96. }
97.
98. public:
99.     void process()
100.    {
101.        computeArrayAandB();
102.        computeArrayCandD();
103.        displayResults();
104.    }
105. };
106.
107. int main()
108. {
109.     ArrayProcessor processor;
110.     processor.process();

```

```

111.     return 0;
112. }
113.

```

6.2 构建测试汇编程序

```

1. # =====
2. # 数组定义和数据段
3. # =====
4. .data
5.     matrix_a:    .space 240      # 数组 A
6.     matrix_b:    .space 240      # 数组 B
7.     matrix_c:    .space 240      # 数组 C
8.     matrix_d:    .space 240      # 数组 D
9.
10. # =====
11. # 代码段
12. # =====
13. .text
14.     j startup
15.     exception_handler:
16.         nop
17.         j exception_handler
18.
19. startup:
20.     # 初始化寄存器
21.     li $t0, 0          # 数组 A 元素值
22.     li $t1, 1          # 数组 B 元素值
23.     li $t2, 0          # 数组 C 元素值
24.     li $t3, 0          # 数组 D 元素值
25.     li $t4, 4          # 偏移量基值
26.     li $t5, 0          # A[i-1]初始值
27.     li $t6, 1          # B[i-1]初始值
28.     li $t7, 0          # 条件标志
29.     li $t8, 240        # 循环终止条件
30.     li $t9, 3          # 乘法常数 3
31.
32.     # 初始化数组 A[0]
33.     li $s0, 0
34.     la $s1, matrix_a
35.     sw $t0, 0($s1)
36.
37.     # 初始化数组 B[0]
38.     la $s1, matrix_b
39.     sw $t0, 0($s1)

```

```

40.
41.     # 初始化数组 D[0]
42.     li $s2, 1
43.     la $s1, matrix_d
44.     sw $t1, 0($s1)
45.
46. processing_loop:
47.     # 计算当前索引
48.     srl $s3, $t4, 2
49.
50.     # 计算 A[i] = A[i-1] + (i>>2)
51.     addu $t5, $t5, $s3
52.     la $s1, matrix_a
53.     addu $s1, $s1, $t4
54.     sw $t5, 0($s1)
55.
56.     # 计算 B[i] = B[i-1] + 3*(i>>2)
57.     mul $s4, $t9, $s3
58.     addu $t6, $t6, $s4
59.     la $s1, matrix_b
60.     addu $s1, $s1, $t4
61.     sw $t6, 0($s1)
62.
63.     # 条件分支 1: 前 20 个元素
64.     slti $t7, $t4, 80
65.     beq $t7, $zero, condition_2
66.
67.     # 前 20 个元素的处理
68.     la $s1, matrix_d
69.     addu $s1, $s1, $t4
70.     sw $t6, 0($s1)
71.
72.     # 保存中间值
73.     move $s5, $t5
74.     move $s6, $t6
75.     j condition_end
76.
77. condition_2:
78.     # 条件分支 2: 中间 20 个元素
79.     slti $t7, $t4, 160
80.     beq $t7, $zero, condition_3
81.
82.     # C[i] = A[i] + B[i]
83.     addu $s5, $t5, $t6

```

```

84.    la $s1, matrix_c
85.    addu $s1, $s1, $t4
86.    sw $s5, 0($s1)
87.
88.    # D[i] = A[i] * B[i]
89.    mul $s6, $t5, $t6
90.    la $s1, matrix_d
91.    addu $s1, $s1, $t4
92.    sw $s6, 0($s1)
93.
94.    j condition_end
95.
96. condition_3:
97.    # 条件分支 3: 后 20 个元素
98.    # C[i] = A[i] * B[i]
99.    mul $s5, $t5, $t6
100.    la $s1, matrix_c
101.    addu $s1, $s1, $t4
102.    sw $s5, 0($s1)
103.
104.    # D[i] = C[i] * B[i]
105.    mul $s6, $s5, $t6
106.    la $s1, matrix_d
107.    addu $s1, $s1, $t4
108.    sw $s6, 0($s1)
109.
110. condition_end:
111.    # 循环控制
112.    addiu $t4, $t4, 4
113.    bne $t4, $t8, processing_loop
114.
115.    # =====
116.    # 程序结束
117.    # =====
118. program_exit:
119.    nop
120.    j program_exit

```

6.3 验证结果

i=56	0000063c	000012b5	0074a06c	85bd045c
i=57	00000675	00001360	007d1ae0	77e8b400
i=58	000006af	0000140e	00860992	8013edfc
i=59	000006ea	000014bf	008f7096	9fd0b7ea

七、实验验算程序下板测试过程与实现

7.1 管脚约束

```
1. # =====
2. # 时钟约束
3. # =====
4. set_property PACKAGE_PIN E3 [get_ports clk]
5. set_property IOSTANDARD LVCMOS33 [get_ports clk]
6. create_clock -period 100.000 -name clk_pin -waveform {0.000
   50.000} [get_ports clk]
7.
8. # =====
9. # 按键和开关约束
10. # =====
11. # 复位按键
12. set_property PACKAGE_PIN N17 [get_ports rst]
13. set_property IOSTANDARD LVCMOS33 [get_ports rst]
14.
15. # 使能按键
16. set_property PACKAGE_PIN V10 [get_ports ena]
17. set_property IOSTANDARD LVCMOS33 [get_ports ena]
18.
19. # 拨码开关 (3 位)
20. set_property PACKAGE_PIN M13 [get_ports {switch[2]}]
21. set_property PACKAGE_PIN L16 [get_ports {switch[1]}]
22. set_property PACKAGE_PIN J15 [get_ports {switch[0]}]
23. set_property IOSTANDARD LVCMOS33 [get_ports {switch[2]}]
24. set_property IOSTANDARD LVCMOS33 [get_ports {switch[1]}]
25. set_property IOSTANDARD LVCMOS33 [get_ports {switch[0]}]
26.
27. # =====
28. # 七段数码管段选约束
29. # =====
30. set_property PACKAGE_PIN T10 [get_ports {o_seg[0]}]
31. set_property PACKAGE_PIN R10 [get_ports {o_seg[1]}]
32. set_property PACKAGE_PIN K16 [get_ports {o_seg[2]}]
33. set_property PACKAGE_PIN K13 [get_ports {o_seg[3]}]
34. set_property PACKAGE_PIN P15 [get_ports {o_seg[4]}]
35. set_property PACKAGE_PIN T11 [get_ports {o_seg[5]}]
36. set_property PACKAGE_PIN L18 [get_ports {o_seg[6]}]
37. set_property PACKAGE_PIN H15 [get_ports {o_seg[7]}]
38.
39. set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[7]}]
40. set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[6]}]
```

```

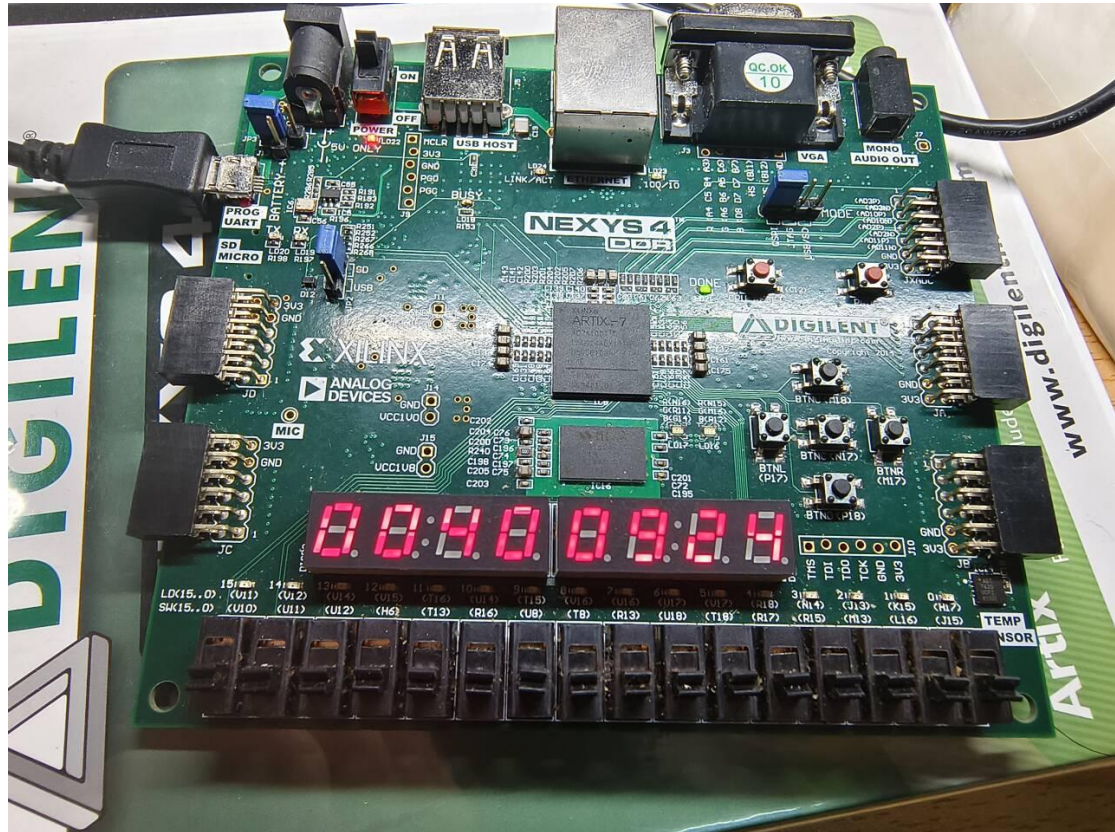
41. set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[5]}]
42. set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[4]}]
43. set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[3]}]
44. set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[2]}]
45. set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[1]}]
46. set_property IOSTANDARD LVCMOS33 [get_ports {o_seg[0]}]
47.
48. # =====
49. # 七段数码管位选约束
50. # =====
51. set_property PACKAGE_PIN J17 [get_ports {o_sel[0]}]
52. set_property PACKAGE_PIN J18 [get_ports {o_sel[1]}]
53. set_property PACKAGE_PIN T9 [get_ports {o_sel[2]}]
54. set_property PACKAGE_PIN J14 [get_ports {o_sel[3]}]
55. set_property PACKAGE_PIN P14 [get_ports {o_sel[4]}]
56. set_property PACKAGE_PIN T14 [get_ports {o_sel[5]}]
57. set_property PACKAGE_PIN K2 [get_ports {o_sel[6]}]
58. set_property PACKAGE_PIN U13 [get_ports {o_sel[7]}]
59.
60. set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[7]}]
61. set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[6]}]
62. set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[5]}]
63. set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[4]}]
64. set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[3]}]
65. set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[2]}]
66. set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[1]}]
67. set_property IOSTANDARD LVCMOS33 [get_ports {o_sel[0]}]
68.
69. # =====
70. # 时序约束
71. # =====
72. # 复位信号的输入延迟约束
73. set_input_delay -clock [get_clocks clk_pin] 1.000 [get_ports rst]
74.
75. # 所有输出端口的输出延迟约束
76. set_output_delay -clock [get_clocks clk_pin] 0.000 [get_ports -
    filter {DIRECTION == "OUT"}]
77.
78. # =====
79. # 可选：添加上拉或下拉约束（如果需要）
80. # =====
81. # 为按键和开关添加上拉，避免悬空时的噪声
82. set_property PULLUP TRUE [get_ports rst]
83. set_property PULLUP TRUE [get_ports ena]

```

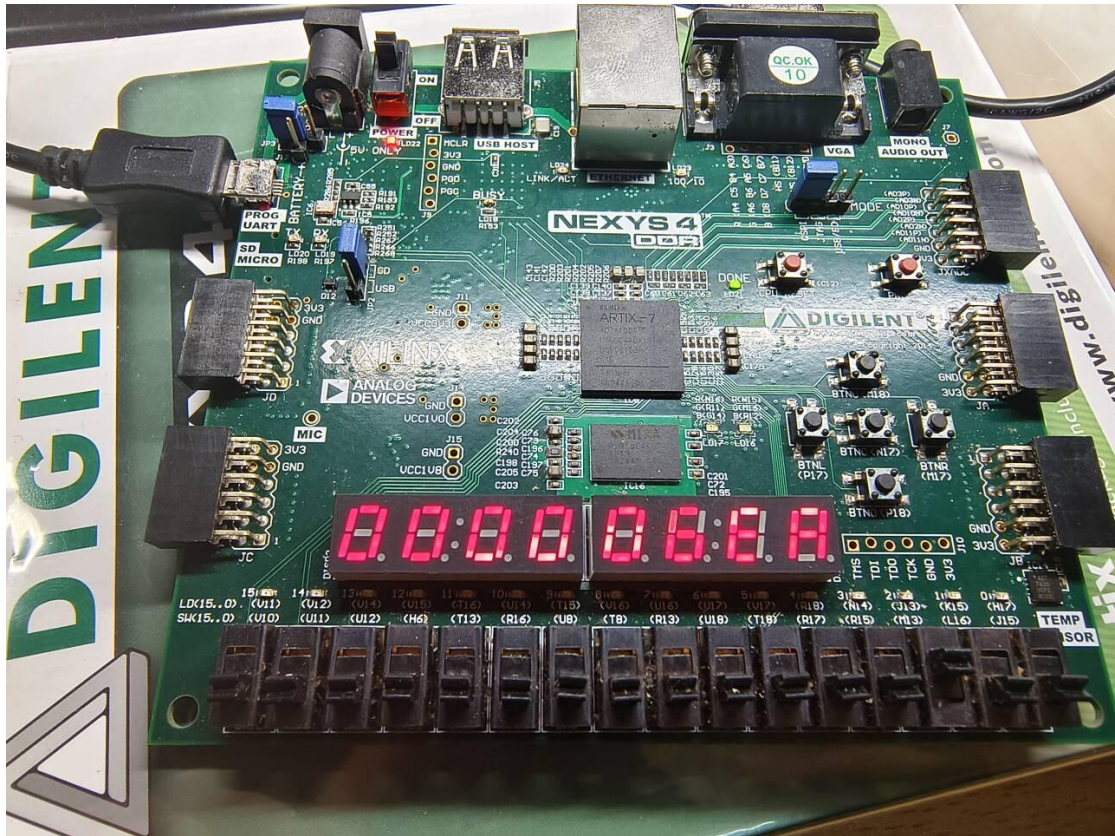
```
84. # set_property PULLUP TRUE [get_ports {switch[*]}]
```

7.2 下板结果说明

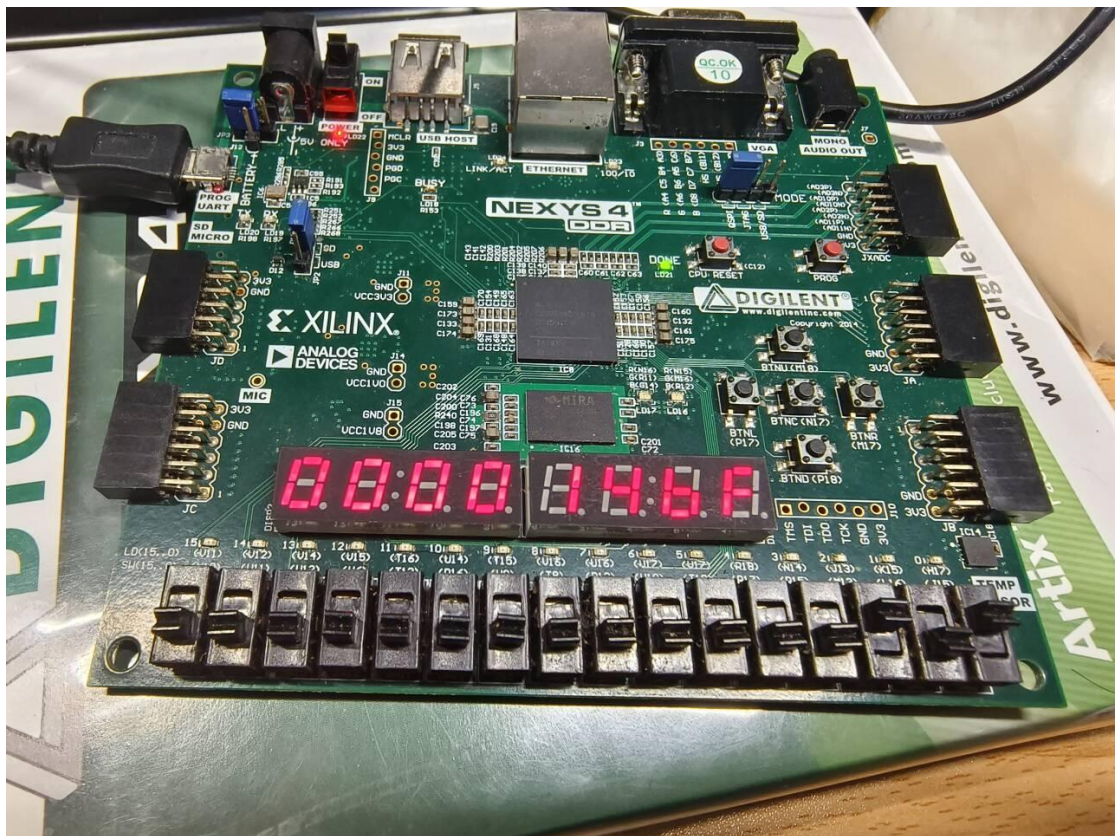
Pc 的值（已结束）



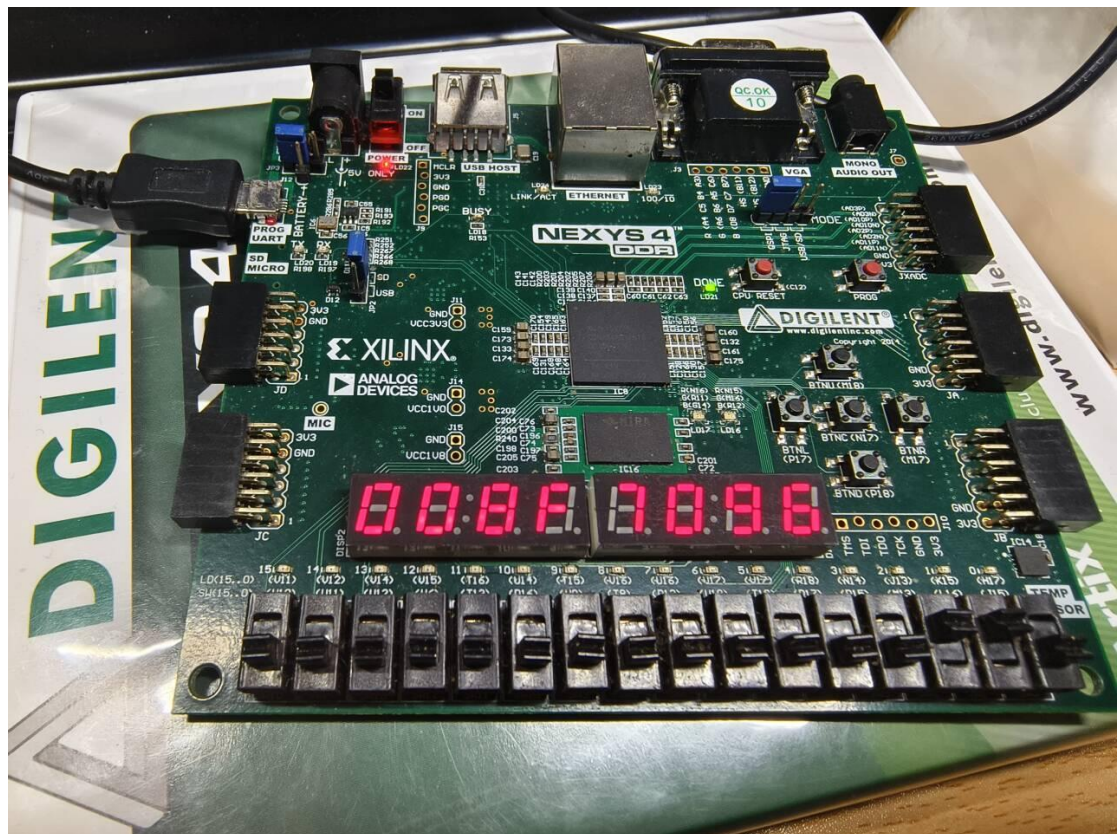
寄存器 a 0000 06EA



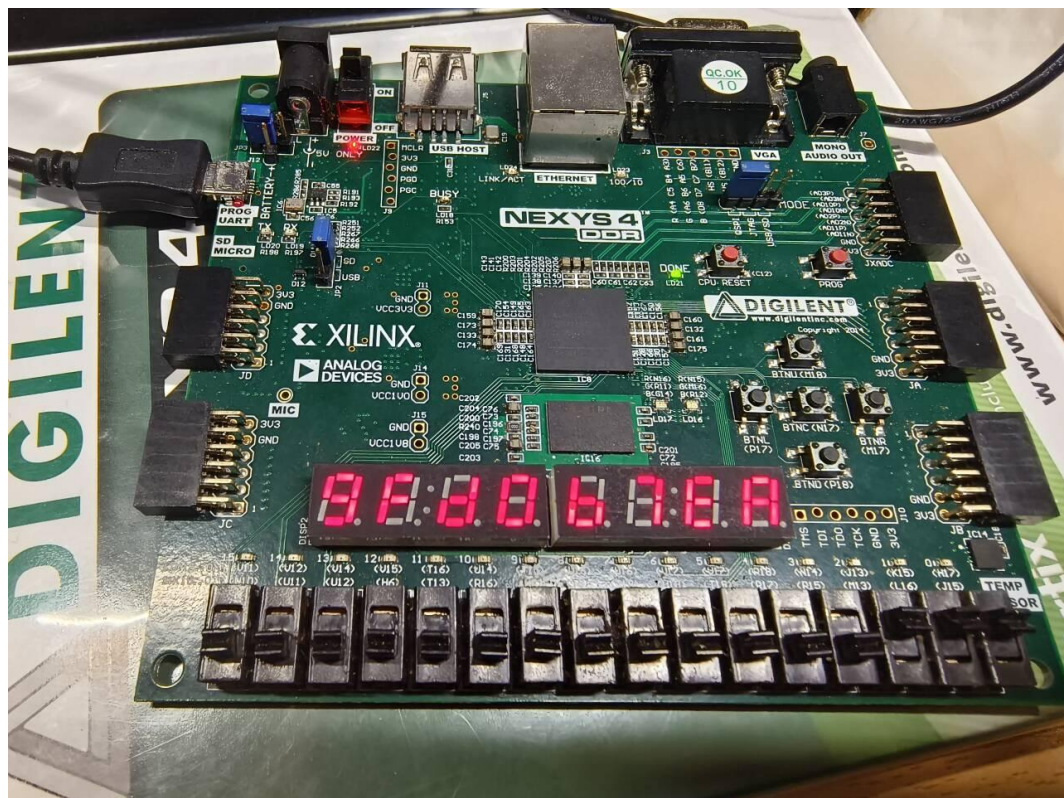
寄存器 b 0000 14bf



寄存器 c 00EF 7096



寄存器 d 9FD0 67EA



八、流水线的性能指标定性分析

8.1 吞吐率 (Throughput, TP)

吞吐率是指在单位时间内流水线所完成的指令数量。它是衡量流水线处理能力的重要指标。

计算公式：

$$TP = \frac{\text{有效指令总数}}{\text{执行总时间}}$$

计算过程：

在该程序段中，流水线在 2052 个时钟周期内实际完成了 1984 条有效指令。

$$TP = \frac{1984}{2052} \approx 0.967 \text{ 条/时钟周期}$$

8.2 加速比 (Speedup Ratio, S)

加速比描述了使用流水线技术后，相比于非流水线（顺序执行）系统在速度上的提升倍数。

计算公式：

$$S = \frac{\text{单周期顺序执行所需时间}}{\text{流水线执行所需时间}} = \frac{n \times k}{T_{\text{total}}}$$

计算过程：

若采用非流水线结构，每条有效指令需完整执行 5 个周期。

$$T_{\text{non-pipe}} = 1984 \times 5 = 9920 \text{ 周期}$$

$$S = \frac{9920}{2052} \approx 4.834$$

8.3 效率 (Efficiency, E)

效率是指流水线中各设备（段）的时间利用率，即有效时空区与总时空区的比值。

计算公式：

$$E = \frac{\text{有效指令占用的时空区}}{\text{总时空区}} = \frac{n \times k}{T_{\text{total}} \times k}$$

计算过程：

$$E = \frac{1984 \times 5}{2052 \times 5} = \frac{9920}{10260} \approx 0.967$$

九、总结与体会

9.1 理论与工程实践的深度融合

通过本次“动态流水线 CPU 设计与性能分析”实验，我深刻体会到了计算机系统结构课程

中理论知识与工程实践相结合的丰富内涵。从最初对流水线抽象概念的理解，到最终在 Xilinx NEXYS4 DDR 开发板上实现完整的硬件系统，这一过程让我对现代处理器的内部工作机制有了更加直观和深入的认识。

9.2 流水线冲突处理的挑战与对策

实验过程中，我最大的收获是理解了流水线技术在带来性能优势的同时所面临的挑战：

- **理想与现实的差距：**五级流水线理论上可将吞吐率提升至单周期的 5 倍，但实际执行中必须面对数据相关和控制相关带来的冲突。
- **冲突解决机制：**通过集成旁路前推单元 (Forwarding Unit) 解决了大部分 RAW 相关，并利用流水线暂停逻辑 (Stall Unit) 处理 Load-use 等无法前推的冲突。
- **工程思维的锻炼：**这种“发现冲突—分析类型—硬件实现解决”的过程，是极其宝贵的工程思维训练。

9.3 硬件描述语言与仿真调试心得

在使用 Verilog HDL 和 Vivado 2016.2 开发环境的过程中，我认识到硬件开发与软件编程的本质区别：

- **时序与组合逻辑：**硬件设计要求极其严谨的时序控制，级间寄存器作为流水线的骨干，确保了指令在不同阶段的同步流动。
- **仿真工具的重要性：**通过 Vivado 内置仿真器观察波形图，我能够直观地分析信号在时钟边沿触发下的传播延迟，这对于调试复杂的数据前递机制至关重要。

9.4 软硬件协同验证与性能定量分析

实验采用了“数组处理器”数学模型进行验证，实现了从 C 语言算法到 MIPS 汇编再到硬件运行的完整链路：

- **结果一致性：**当七段数码管动态显示出与 C 语言参考程序一致的十六进制结果（如寄存器 d 最终显示为 9FD0 67EA）时，验证了处理器逻辑的正确性。
- **性能量化反思：**通过对测试程序执行数据的分析，得出吞吐率 0.967 条/时钟周期，加速比 4.834。

十、 源代码（见附件）